SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Programme: **B.E / B.Tech**

Course Code: **CSA04**

Course Name: **Operating Systems**

Duration: **1: 30 Hrs.** ·

Date : Max Marks: **50 Marks**

CONTINUOUS INTERNAL ASSESSMENT 1

Course Outcomes

On successful completion of the course, students will be able to:

CO	Course Outcome Statement
CO1	Apply the concepts of Operating System types, structures, and system calls to demonstrate their functions in various computing environments
CO2	Analyze process management and deadlock handling techniques to improve CPU utilization and system performance

Answer all the Questions

1) A university wants to replace its old batch-processing computer with a modern setup. Using the evolution of operating systems, illustrate how the OS would progress from the batch system through multiprogramming and time-sharing to the present day. Apply these stages to show how CPU utilization and response time improve at each step.

(10)(CO1) (BTL3)

2) A company is setting up a lab of desktop computers and a high-performance server. Apply your knowledge of desktop systems and multiprocessor systems to compare symmetric and asymmetric multiprocessing. Show with an example how the OS schedules tasks on a multiprocessor system, and explain the advantages gained over a single-processor desktop.

(10)(CO1) (BTL3)

3) a) A student runs a program that copies the contents of a source file to a destination file. Apply your knowledge of system calls to illustrate the sequence of system calls the program makes, classify the system calls into their major types with one example each, and explain how a system call moves from the user program to the kernel.

(5)(CO1) (BTL3)

b) Five processes arrive in the ready queue as follows:

Process	Arrival Time	Burst Time	Priority Value
P1	0	6	3
P2	1	4	1
P3	2	5	4
P4	3	2	2
P5	4	3	5

Draw the Gantt charts for FCFS and for non-preemptive priority scheduling (lower number = higher priority). Calculate the waiting time and turnaround time of each process with the averages, deduce which algorithm gives the minimum average waiting time, and identify the problem of starvation and its remedy.

(5)(CO2) (BTL4)

4) A system has three resource types A (7 instances), B (2 instances) and C (6 instances), and five processes:

Process	Allocation (A B C)	Request (A B C)
P0	0 1 0	0 0 0
P1	2 0 0	2 0 2
P2	3 0 3	0 0 0
P3	2 1 1	1 0 0
P4	0 0 2	0 0 2

(a) Compute the Available vector. *(2 mark)*

(b) Apply the deadlock detection algorithm and show that the system is not deadlocked, giving a completion sequence. *(4 marks)*

(c) Now suppose P2 makes an additional request of (0 0 1). Analyze the new state and identify the deadlocked processes. *(4 marks)*

(10) (CO2) (BTL4)

5)) a) Two processes share counter = 10. The producer executes counter++ and the consumer executes counter--. Each is compiled to three steps: load into a register, modify the register, store back. The following interleaving occurs:

Time	Process	Operation
T0	Producer	R1 = counter
T1	Producer	R1 = R1 + 1
T2	Consumer	R2 = counter
T3	Consumer	R2 = R2 − 1
T4	Producer	counter = R1
T5	Consumer	counter = R2

Calculate the final value of counter. Analyze the set of all possible final values and state the correct value. *(4 marks)*

b) A readers-writers solution (readers' priority) uses mutex = 1, wrt = 1 and readcount = 0. The first reader executes wait(wrt) and the last reader executes signal(wrt). The events occur in this order: R1 arrives, R2 arrives, W1 arrives, R3 arrives, R1 leaves, R2 leaves, R3 leaves. Trace the values of readcount and wrt after each event and identify which processes are blocked. Analyze the starvation risk. *(4 marks)*

c) Five philosophers share five chopsticks, and each need two to eat. Calculate the maximum number of philosophers who can eat at the same time. Examine how deadlock arises if all pick up the left chopstick simultaneously, and how limiting the table to 4 philosophers resolves it. *(2 marks)*

(10) (CO2) (BTL4)

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

(Declared as Deemed to be University under section 3 of UGC Act 1956)

ANSWER SCRIPT EVALUATION

CONTINUOUS INTERNAL ASSESSMENT 1

Programme	**B.E. / B. Tech.**	**Course Code**	**CSA04**
Course Name	**Operating Systems**	**Date**	
Duration	**1 Hr 30 Mins**	**Max. Marks**	**50**

Q. No.	**Max. Marks**	**Marks Awarded**
1	10	
2	10	
3(a)	5	
3(b)	5	
4	10	
5	10	
Total Marks	50	

Total in words:

NAME: JOEL BENN JOHN B	COURSE: OPERATING SYSTEM
R.NO: 191126O005	COURSE CODE: CSAO404
DATE: 03-11-26	CIA 1

1.] ## Evolution of Operating Systems:

Introduction:

An Operating system (OS) is system software that manages computer hardware and software resources. It acts as an interface between the user and the computer hardware.

The evolution of OS improved CPU utilization, response time, resource management & user interaction.

① Batch processing System:

A batch OS executes a group of similar jobs together without direct interaction between the user and computer.

Working:

- Users submit jobs to the operator.
- Similar jobs are collected into batches.
- The OS executes jobs one after another.
- Results are produced after execution.

Ex: Payroll processing and electricity bill generation.

Limitations:

- Long turnaround time.
- No direct user interaction.
- CPU remains idle during I/O operations.

Multiprogramming Operating System:

Multiprogramming allows multiple programs to reside in main memory simultaneously.

When one process waits for I/O, the CPU switches to another ready process.

- Working:

1.) Multiple Jobs are loaded into memory.

2.) The CPU executes one process.

3.) When that process waits for I/O, the OS selects another process.

4.) The CPU continues execution without remaining idle.

Ex: A computer executes a calculation program while another program waits for disk input.

Advantages:

- Improved CPU utilization
- Increased throughput.
- Reduced CPU idle time.

- **Time-sharing OS:**

A time-sharing OS allows many users to interact with a computer. The CPU gives each process a small time quantum & switches among them, often using Round Robin

- provides quick response & interactive computing.
- Supports simultaneous users and fair CPU allocation.

Ex: students accessing a university server through terminals.

- **Modern OS:**

Modern OSs combine multiprogramming, time-sharing, multiprocessing, networking, and security.

Ex: Windows, Linux, macOS, Android & iOS.

- Multitasking & multiprocessing.
- GUI, virtual memory, networking/cloud support.
- Security & access control.

2. **Desktop Systems & Multiprocessor Systems:**

A desktop system is a personal computer for individual use. A multiprocessor system contains 2 or more processors working under one OS & sharing memory / resources. It improves performance through parallel execution.

1. **Desktop System:**

Typical components: CPU, RAM, storage, I/O devices & OS.

Ex: A personal computer running Windows.

Advantages: Easy to use, low-cost, suitable for personal work, simple to maintain.

2. **multiprocessor system:**

Multiple processors execute processes simultaneously and share system resources.

Ex: A server handles database queries, student logins, & file services at the same time.

Advantages: Increased throughput, faster execution, improved reliability, better resource use.

3. **Symmetric multiprocessing (SMP):**

In SMP, all processors have equal status & share main memory I/O devices, & one OS.

- Any processor can execute any process.
- A common OS manages scheduling.
- Load balancing is easier.

Ex: Multicore computer running applications and OS tasks on different cores.

4. Asymmetric multiprocessing (AMP):

In AMP, processors have different roles. One master processor controls the system, while slave processors perform assigned tasks.

- Master usually controls scheduling.
- Slaves perform specialized work.
- Used in embedded & specialised systems.

Ex: One processor handles control while another performs signal processing.

Example of Parallel work:

On a 4-processor server: CPU 1 runs a database query, CPU 2 processes files, CPU 3 handles logins, and CPU 4 performs backup. The OS distributes tasks for parallel execution. SMP provides equal processor access, AMP follows a master-slave model. Both improve performance over a single-processor system.

3.] (d) System calls & file copying.

A system call is the interface through which a user program requests services from the OS kernel. It supports process, file, device, communication, information & protection operations.

File-copying sequence:

Open (source.txt) → create (destination.txt) → read (source) → write (destination) ← repeat until EOF ← close both files

Common system calls:

Category	Purpose	Example
Process control	Create / Terminate processes	fork()
File management	Read / write files	read()
Device management	Control devices	ioctl()
Information	Obtain system data	getpid()
Communication	Exchange data	pipe()
protection	Set permissions	chmod()

Working of a System Call:

1. Program calls a library function.
2. System-call number & arguments are passed.

trap instruction transfers control to kernel.

4. CPU changes from user mode to kernel mode.
5. Kernel performs operation & returns result.
6. CPU returns to user mode.

(b) **FCFS & Non-preemptive priority Scheduling:**

Process	AT	BT	priority.
P1	0	6	3
P2	1	4	1
P3	2	5	4
P4	3	2	2
P5	4	3	5

given, lower priority no. means higher priority.

Formulas:

Turnaround Time (TAT) = CT - AT

Waiting Time (WT) = CT - AT - BT = TAT - BT

1. **FCFS:** processes execute in order of arrival.

P1	P2	P3	P4	P5
0	6	10	15	17 20

process	CT	WT	TAT
P1	6	0	6
P2	10	5	9
P3	15	8	13
P4	17	12	14
P5	20	13	16
Total:		38	58

Average WT = 38/5 = 7.6

Average TAT = 58/5 = 11.6

2. Non-preemptive priority.

process	CT	WT	TAT
P1	6	0	6
P2	10	5	9
P3	17	10	15
P4	12	7	9
P5	20	13	16
Total:		35	55

Average WT = 35/5 = 7

Average TAT = 55/5 = 11

Algorithm	Average WT	Average TAT
FCFS	7.6	11.6
priority	7	11

Starvation & Remedy:

Starvation occurs when a low-priority process waits indefinitely. Aging gradually increases its priority as waiting time grows, ensuring it eventually receives CPU time.

4] Deadlock Detection Algorithm:

Deadlock occurs when processes wait indefinitely for resources held by one another. Detection checks available resources, current allocation, outstanding Request.

Total resources : A = 7, B = 2, C = 6

Process	Allocation (A, B, C)	Request (A, B, C)
P0	(0, 1, 0)	(0, 0, 0)
P1	(2, 0, 0)	(2, 0, 2)
P2	(3, 0, 3)	(0, 0, 0)
P3	(2, 1, 1)	(1, 0, 0)
P4	(0, 0, 2)	(0, 0, 2)

(a) **Available vector:**

Total allocated: $A = 0+2+3+2+0 = 7$,

$B = 1+0+0+1+0 = 2$,

$C = 0+0+3+1+2 = 6$.

Available = total − allocated $= (7,2,6) - (7,2,6) = (0,0,0)$.

(b) **Detection Algorithm:**

7. Set work = available & finish[i] = false.
8. Find an unfinished process with request ≤ work.
9. If found, add its allocation to work & mark it finished.
10. Repeat until all finish or no eligible process remains.

Initial work = (0,0,0).

Step	Process	Request	work after completion
1	P0	(0,0,0)	(0,1,0)
2	P2	(0,0,0)	(3,1,3)
3	P3	(1,0,0)	(5,2,4)
4	P1	(2,0,2)	(7,2,4)
5	P4	(0,0,2)	(7,2,6)

Sequence: P0 → P2 → P3 → P1 → P4. All processes finish, hence system is not deadlocked.

P2 Requests additional (0, 0, 1)

Process	Request	can finish?
P0	(0, 0, 0)	Yes
P1	(2, 0, 2)	No
P2	(0, 0, 1)	No
P3	(1, 0, 0)	No
P4	(2, 0, 2)	No

Unfinished processes : P1, P2, P3, P4. They are identified as deadlocked under this resource state.

5.) Producer - consumer Race condition

(a) A race condition occurs when concurrent processes access & modify shared data, & the result depends on execution order.

Time	process	operation
T0	Producer	R1 = counter = 10
T1	producer	R1 = R1 + 1 = 11
T2	Consumer	R2 = counter = 10
T3	Consumer	R2 = R2 - 1 = 9
T4	Producer	counter = R1 = 11
T5	Consumer	counter = R2 = 9.

Use a mutex to protect the critical section:

wait(mutex) → counter++ → signal(mutex)

(b) **Readers – writers problem.**

Multiple readers may access shared data simultaneously. but a writer needs exclusive access.

Given: mutex = 1, wrt = 1, readcount = 0.

Reader algorithm.

wait(mutex); readcount++

if readcount == 1 : wait(wrt)

signal(mutex); READ

wait(mutex); readcount--

if readcount == 0 : signal(wrt)

signal(mutex)

Event	readcount	wrt	Status.
Initial	0	1	Idle
R1 arrives	1	0	R1 reading
R2 arrives	2	0	R1, R2 reading
W1 arrives	2	0	writer blocked
R3 arrives	3	0	R1, R2, R3 reading
R1 leaves	2	0	R2, R3 reading
R2 leaves	1	0	R3 reading
R3 leaves.	0	0	W1 can access

Starvation:

- Continuous arrival of readers may keep a writer waiting indefinitely (writer starvation)
- Remedy: Use a fair solution, writer priority when appropriate, or a FIFO queue (semaphore).

(c) Dining philosophers problem.

5 philosophers sit around a circular table with 5 chopsticks. Each needs 2 chopsticks to eat.

Max. Concurrent Eating:

2 non-adjacent philosophers can eat simultaneously, Maximum = 2.

Deadlock:

If all philosophers pick up their left chopstick together, each waits for right one. This circular wait causes deadlock.

Solution:

Allow at most 4 philosophers to compete for chopsticks at once. This leaves a chopstick available, breaks circular wait, prevents deadlock.